EXPERIMENT NO. 08

TCP Socket Programming – Factorial Calculation

Aim: To write and execute a TCP socket programming client-server application in which the client sends a number to the server, the server calculates its factorial, and sends the result back to the client.

Objectives

- To understand the concept of TCP socket programming.
- To understand client-server communication using TCP.
- To establish a reliable connection between a client and a server.
- To send data from the client to the server using sockets.
- To perform factorial calculation at the server side.
- To send the calculated result back from the server to the client.
- To understand the basic socket functions used in network programming.

Apparatus / Software Required

1. Hardware
   - Computer/Laptop
   - Wired Ethernet connection or LAN network
2. Software
   - Windows/Linux operating system
   - Python 3.x
   - Python IDE / VS Code / IDLE / Terminal
   - Two computers connected through LAN, or one computer using localhost

Theory

Socket Programming

A socket is an endpoint used for communication between two devices over a computer network. Socket programming allows applications running on different computers to communicate with each other.

In a client-server model:

- The server waits for incoming connection requests.
- The client initiates a connection with the server.
- After establishing the connection, both sides can exchange data.

For this experiment, TCP (Transmission Control Protocol) is used.

TCP

TCP is a connection-oriented and reliable transport layer protocol.

Before data transmission begins, TCP establishes a connection between the client and server. It provides:

- Reliable data delivery
- Error detection
- Ordered delivery of data
- Flow control
- Connection-oriented communication
- Retransmission of lost packets

TCP is suitable for applications where reliable communication is important.

Client-Server Model

The program consists of two parts:

Server

The server:

1. Creates a socket.
2. Binds the socket to an IP address and port number.
3. Listens for incoming connections.
4. Accepts the client connection.
5. Receives a number from the client.
6. Calculates its factorial.
7. Sends the result back to the client.
8. Closes the connection.

Client

The client:

1. Creates a socket.
2. Connects to the server.
3. Takes a number from the user.
4. Sends the number to the server.
5. Receives the factorial from the server.
6. Displays the result.
7. Closes the connection.

Important Socket Functions

Function, Description

socket(), Creates a socket

bind(), Associates the server socket with an IP address and port

listen(), Places the server in listening mode

accept(), Accepts an incoming client connection

connect(), Establishes a connection with the server

send(), Sends data through the socket

recv(), Receives data through the socket

close(), Closes the socket connection

Factorial

The factorial of a non-negative integer n is represented by $n!$

It is calculated as:

$n! = n \times (n-1) \times (n-2) \times \ldots \times 2 \times 1$

For example:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Also:

$0! = 1$

In this experiment, the server performs the factorial calculation.

Algorithm

Server Algorithm

1. Start.
2. Import the socket module.
3. Create a TCP socket using socket().
4. Bind the socket to an IP address and port number.
5. Start listening for client connections.
6. Accept the client connection.
7. Receive the number from the client.
8. Convert the received data into an integer.
9. Calculate the factorial of the number.
10. Convert the result into a string and send it to the client.
11. Close the connection.
12. Stop.

Client Algorithm

1. Start.
2. Import the socket module.
3. Create a TCP socket.
4. Connect to the server using its IP address and port number.
5. Accept a number from the user.
6. Send the number to the server.
7. Receive the factorial result from the server.
8. Display the result.
9. Close the socket.
10. Stop.

Program

The following implementation uses Python TCP sockets.

A. Server Program

Save the file as `server.py`

Factorial > server.py > ...

```python
import socket
import math

# Create TCP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Server IP and port
HOST = "0.0.0.0"
PORT = 5000

# Bind socket
server_socket.bind((HOST, PORT))

# Listen for client connection
server_socket.listen(1)

print("Server started...")
print("Waiting for client connection...")

# Accept client connection
conn, addr = server_socket.accept()

print("Connected to client:", addr)

# Receive number from client
data = conn.recv(1024).decode()

number = int(data)

print("Number received from client:", number)

# Calculate factorial
if number < 0:
    result = "Factorial is not defined for negative numbers."
else:
    result = str(math.factorial(number))

# Send result to client
conn.send(result.encode())

print("Factorial sent to client:", result)

# Close connection
conn.close()
server_socket.close()

print("Connection closed.")
```

B. Client Program

Save the file as `client.py`

Factorial > client.py > ...

```python
import socket

# Create TCP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Server IP and port
HOST = "127.0.0.1"
PORT = 5000

# Connect to server
client_socket.connect((HOST, PORT))

# Take input from user
number = input("Enter a number: ")

# Send number to server
client_socket.send(number.encode())

# Receive factorial result
result = client_socket.recv(1024).decode()

# Display result
print("Factorial received from server:", result)

# Close connection
client_socket.close()
```

Output

Server Side

```
PS C:\Users\lenovo\Documents\CN Exp_TE\Factorial> python server.py
Server started...
Waiting for client connection...
Connected to client: ('127.0.0.1', 54369)
Number received from client: 4
Factorial sent to client: 24
Connection closed.
```

Client Side

```
PS C:\Users\lenovo\Documents\CN Exp_TE\Factorial> python client.py
Enter a number: 4
Factorial received from server: 24
```

Result

The TCP client-server program was successfully implemented. The client sent a number to the server using TCP socket communication, the server calculated its factorial, and the calculated result was successfully sent back to the client.

Conclusion

Thus, a TCP socket-based client-server application was successfully implemented for a wired network. The experiment demonstrated reliable communication between the client and server using TCP. The server received the number from the client, calculated its factorial, and returned the result to the client successfully. This experiment helps in understanding TCP connection establishment, socket communication, data transmission, and client-server architecture.